LLaMA-Berry: Pairwise Optimization for Olympiad-level Mathematical Reasoning via O1-like Monte Carlo Tree Search

Di Zhang[1,2*], **Jianbo Wu**[3*], **Jingdi Lei**[2*], **Tong Che**[4*]
Jiatong Li[5], **Tong Xie**[6], **Xiaoshui Huang**[7], **Shufei Zhang**[2], **Marco Pavone**[8]
Yuqiang Li[2†], **Wanli Ouyang**[2], **Dongzhan Zhou**[2†]
[1]Fudan University, [2]Shanghai Artificial Intelligence Laboratory, [3]University of California, Merced
[4]Independent Researcher, [5]Hong Kong Polytechnic University, [6]University of New South Wales
[7]Shanghai Jiao Tong University, [8]Stanford University
{liyuqiang,zhoudongzhan}@pjlab.org.cn

arXiv:2410.02884v2 [cs.AI] 21 Nov 2024

Abstract

This paper presents an advanced mathematical reasoning framework, LLaMA-Berry, for enhancing the problem-solving ability of large language models (LLMs). The framework combines Monte Carlo Tree Search with Self-Refine (SR-MCTS) to optimize the reasoning paths and utilizes a pairwise reward model to evaluate different paths globally. By leveraging the self-critique and rewriting capabilities of LLMs, our SR-MCTS overcomes the inefficiencies and limitations of conventional stepwise and greedy search algorithms by fostering a more efficient exploration of solution spaces. To guide the search process, we propose Pairwise Preference Reward Model (PPRM) to predict pairwise preferences between solutions through instruction-following capabilities trained by Reinforcement Learning from Human Feedback (RLHF). Finally, the Enhanced Borda Count (EBC) method is adopted to synthesize pairwise preferences into global quantile scores for evaluations. This approach addresses the challenges of scoring variability and non-independent distributions in mathematical reasoning tasks. The framework has been tested on general and advanced benchmarks, showing superior search efficiency and performance compared to existing open-source and closed-source methods, particularly in complex Olympiad-level benchmarks, including AIME24 and AMC23.

1 Introduction

Mathematical reasoning represents a great challenge in artificial intelligence, with broad applications across automated theorem proving, mathematical problem solving, and scientific discovery (Ahn et al., 2024). Recently, significant strides have been made by large language models (LLMs) like GPT-4 (Achiam et al., 2023) in general mathematical tasks involving arithmetic and geometric problem-solving (Cobbe et al., 2021; Sun et al., 2024; Ying et al., 2024). However, complex mathematical reasoning remains challenging, especially at the Olympiad-level benchmarks such as AIME (MAA, 2024).

An intuitive approach to improving problem-solving is to break solutions into step-by-step reasoning paths (Lightman et al., 2023; Luo et al., 2024a), as demonstrated in Chain-of-Thought (CoT Wei et al., 2022). While prompt-based methods can effectively facilitate the construction of such reasoning paths, they may still encounter challenges due to the lack of comprehensive feedback during the generation process, which can affect efficiency (Paul et al., 2023). In contrast to stepwise generation methods, another promising line of research treats the entire solution as an independent state, employing rewriting capabilities to refine the solutions, such as in Self-Refine (Madaan et al., 2023a) and Reflexion (Shinn et al., 2024). However, these approaches, while innovative, may occasionally face challenges like being susceptible to local optima or potentially drifting towards suboptimal solutions due to flawed feedback, which could impact their maximum potential performance.

In addition to generating reasoning paths, effective solution evaluation is crucial, with models like the outcome reward model (ORM) and process reward model (PRM) (Uesato et al., 2022) serving as valuable examples. The ORM focuses on the correctness of the final answer in a reasoning path, while the PRM emphasizes the correctness of each step in the process. While both methods enable reward models to assign scalar scores, obtaining reliable labeled data for training these reward models remains a significant challenge. Moreover, the scoring standards for mathematical reasoning tasks can vary significantly, as each problem presents unique characteristics. This variation complicates the scaling of reward models and hinders their abil-

*These authors contributed equally.
†Corresponding author

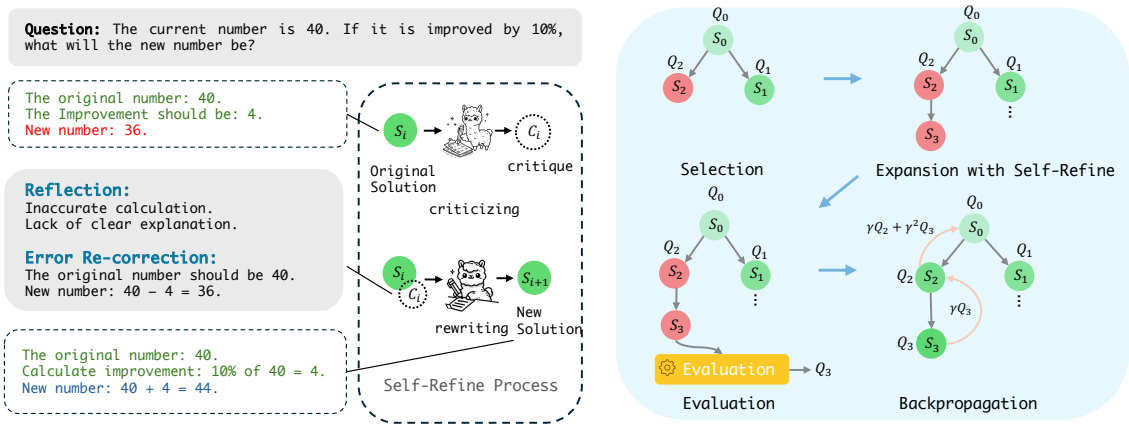

Figure 1: The main pipeline of LLaMA-Berry, where S_i stand for problem-solving solutions and C_i stands for critiques. The pipeline consists of four phases detailed in Section 2.2, including selection, expansion, evaluation, and backpropagation.

ity to capture local preference relations between solutions. Although trained using language models, these reward models have yet to fully leverage instruction-following capabilities, which may limit their effectiveness in handling more complex reasoning tasks (Zhang et al., 2024a).

To improve the efficiency of solution search in mathematical problems, we treat a complete solution as an independent state and apply Self-Refine to optimize previous solutions in order to obtain better ones. In the Self-Refine process, feedback from critiques is utilized to make the search more efficient compared to stepwise reasoning path generation. Furthermore, we incorporate Monte Carlo Tree Search (MCTS Kocsis and Szepesvári, 2006) to replace the iterative manner in naive Self-Refine, enhancing the solution search. MCTS leverages signals from the evaluation process to assess solutions and uses the Upper Confidence Bound applied to Trees (UCT) method to balance exploration and exploitation. This approach enables the search process to effectively exploit higher-quality solutions and explore those with greater potential for improvement, while avoiding getting trapped in suboptimal local minima.

In evaluation process, utilizing the instruction-following capabilities trained by Reinforcement Learning from Human Feedback (RLHF Christiano et al., 2017), Pairwise Preference Reward Model (PPRM) transforms the absolute rewards calculation into preferences prediction between solutions to calculate rewards. The approach reduces the variability with scoring characteristics and thus leads to a more robust and consistent evaluation of different solutions. To overcome the locality limitations inherent in pairwise comparisons, we employ the Enhanced Borda Count (EBC) method to aggregate local preference evaluations into global quantile scores, leading to more informed decision-making and, ultimately, better solutions. Combining the PPRM and EBC method not only enables the reward model to learn a more robust reward signal but also captures the global characteristics of the solution space, ensuring more reliable comparisons.

Our contributions are summarized as follows: (1) We propose **SR-MCTS**, a novel Markov Decision Process (MDP) framework that treats entire solutions as states and Self-Refine as optimization action to perform advanced solution search with MCTS. (2) **PPRM** is developed to leverage the preference relationship between solutions to evaluate their quality, which avoids the volatility of absolute scores while providing a more guided exploration of optimal paths. We adopt the EBC method to convert the local preferences into global evaluations. (3) We verify the effectiveness of LLaMA-Berry on multiple benchmarks, which outperforms baseline approaches like ToT (Yao et al., 2024) and rStar (Qi et al., 2024) in both search efficiency and accuracy. Notably, LLaMA-Berry enhances the performance of LLaMA-3.1-8B, making the 8B-level model comparable to proprietary models, including GPT-4 Turbo on Olympiad-level mathematical reasoning **without additional training**.

2 Methodology

2.1 Preliminary

One of the core challenges in mathematical problem-solving is to generate and optimize reasoning paths to derive high-quality solutions. We formalize this process in a path-wise Markov Decision Process (MDP) framework, where each state s in the state space S represents a *complete solution* to a given problem, and the action space A consists of all feasible *rewriting* actions a that make transitions between states.

In the framework, we aim to quantify the expected reward $Q(s, a)$ from executing action a at state s, that is,

$$Q(s, a) = \mathbb{E}[R(s')|s' = T(s, a)] \tag{1}$$

, where $T(s, a)$ indicates the transition from s to another solution s' via the rewriting action a. Our primary objective is to identify the optimal state s^* that represents the best solution. We can reach s^* by selecting actions that maximize the reward, guiding us toward the most desirable outcome, as demonstrated in Equation 2.

$$s^* = \arg\max_{s' \in S} Q(s') \tag{2}$$

2.2 Self-Refine applied to MCTS

As shown in Figure 1, SR-MCTS integrates Monte Carlo Tree Search (MCTS) with the Self-Refine mechanism to continuously evaluate and optimize the solution search. This integration leverages the iterative nature of MCTS and the self-improvement capabilities of LLMs, thereby improving the search outcomes.

Monte Carlo Tree Search (MCTS) is an effective method within the Markov Decision Processes (MDP) framework, employing states, actions, and value functions through sampling. The algorithm follows four key steps: selection, expansion, evaluation, and backpropagation. In the selection phase, the root node is expanded using the Upper Confidence Bound applied to Trees (UCT) algorithm, which selects a node s by balancing exploration and exploitation:

$$a = \arg\max_{a \in A(s)} \left(Q(s, a) + c \cdot \sqrt{\frac{\ln N(s)}{N(s, a)}} \right) \tag{3}$$

, where $N(s)$ is the visitation count of node s, $N(s, a)$ is the action frequency, and c is a parameter controlling exploration. In the expansion phase, node s generates subsequent states s', added as new nodes in the tree $\mathcal{T}$. The evaluation phase typically uses simulations or heuristics to estimate the Q-values for these nodes. Finally, during backpropagation, the estimated Q-values are updated retroactively from the leaf nodes to the root. This iterative process allows MCTS to refine decision-making by balancing the exploration of new paths with the exploitation of known high-value paths.

Selection phase. The selection phase identifies a node s_i from the search tree $\mathcal{T}$ for expansion, where each node represents a complete solution state. The Upper Confidence Bound applied to Trees (UCT) algorithm is employed to select the optimal node, with dynamic pruning used to avoid local optima. A node s_i is considered fully explored when its child nodes reach a predefined limit, and at least one child node's Q value exceeds the Q value of s_i.

Expansion phase. In the expansion phase, as shown in Figure 1, the selected answer s_i is expanded by generating successor answers through a Self-Refine process, which includes a **Criticizing** and **Rewriting** process. The **Criticizing** process generates a critique $c_i = C(s_i)$ that identifies drawbacks (e.g., mathematical wrongs or logical faults) in the current chosen answer S_i, and then **Rewriting** process generates a new answer $s_{i+1} = R(s_i, c_i)$. In practice, to simplify the problem, we assume this process is deterministic, ensuring that the same original state of solutions s_i consistently produces the same successor state of solution s_{i+1}. The new state of solution s' is then added to the search tree $\mathcal{T}$ as a new node.

Evaluation phase. The evaluation phase calculates the value $Q(s')$ of the newly generated node s' using the Pairwise Preference Reward Model (PPRM). The evaluation involves two steps: global and local value assessments. The global value $Q_g(s')$ is determined by the quantile of s' in a win-loss preference matrix $\mathbf{M}$, which reflects the win-loss relationships between nodes. The local value $Q_l(s')$ is derived from comparisons with adjacent nodes in the search tree $\mathcal{T}$. The total value $Q(s')$ is then computed as a weighted combination of global and local values: $Q(s') = \alpha Q_g(s') + (1 - \alpha) Q_l(s')$, where α controls the relative influence of each component.

Backpropagation phase. In the backpropagation phase, the value $Q(s')$ of the new node is propagated back to its parent node s_i, updating Q value